LAB 12: ALGORITHMS WITH AI ASSISTANCE

Sorting, Searching, and Optimizing Algorithms

Name: D.POORNANDA CHALAM **Roll No:** 2503A52361 **Batch:** 13

Task 1 – Merge Sort Implementation

Question / Objective:
Write merge_sort(arr) to sort ascending, include time and space complexity in the docstring, and verify with test cases.

AI Prompt:
"Generate a documented merge_sort(arr) with a merge helper. Include complexity and test empty, single-item, duplicate, and unsorted lists."

Source Code:

```
def merge_sort(arr):
    """Sort ascending. Time: O(n log n) in all cases; extra space: O(n)."""
    if len(arr) <= 1:
        return arr[:]
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)


def merge(left, right):
    """Merge two sorted lists."""
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result


if __name__ == "__main__":
    for values in ([38, 27, 43, 3, 9, 82, 10], [], [5], [4, 2, 2, 1]):
        print("Input:", values)
        print("Sorted:", merge_sort(values))
```

Terminal Output (VS Code Execution):

```
[Running] python -u "c:\Users\drona\Documents\AI_Assisted_Coding\Lab12\task1.py"
Input: [38, 27, 43, 3, 9, 82, 10]
```

```
Sorted: [3, 9, 10, 27, 38, 43, 82]
Input: []
Sorted: []
Input: [5]
Sorted: [5]
Input: [4, 2, 2, 1]
Sorted: [1, 2, 2, 4]

[Done] exited with code=0 in 0.168 seconds
```

Justification & Technical Explanation:
Merge Sort applies a divide-and-conquer strategy: recursively bisecting the collection until single-element partitions are reached, then merging them in sorted order. Time complexity is guaranteed O(n log n) across best, average, and worst cases because the bisection depth is log n and each level requires O(n) comparisons. Auxiliary space is O(n) for temporary merge buffers. Edge cases including empty lists, single items, and duplicates were successfully verified.

Task 2 – Binary Search with AI Optimization

Question / Objective:
Create binary_search(arr, target), returning the index or -1. Explain best, average, and worst-case complexities and test different inputs.

AI Prompt:
"Implement iterative binary search on an ascending sorted list, return index or -1, document complexities, and test found and missing targets."

Source Code:

```
def binary_search(arr, target):
    """Search an ascending sorted list; return index or -1.
    Best time: O(1); average/worst time: O(log n); space: O(1).
    The input must already be sorted.
    """
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = low + (high - low) // 2
        if arr[mid] == target:
            return mid
        if arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1


if __name__ == "__main__":
    numbers = [3, 8, 12, 17, 25, 31, 42]
    for target in [17, 3, 42, 99]:
        print(f"Target {target}: index {binary_search(numbers, target)}")
```

Terminal Output (VS Code Execution):

```
[Running] python -u "c:\Users\drona\Documents\AI_Assisted_Coding\Lab12\task2.py"
```

```
Target 17: index 3
Target 3: index 0
Target 42: index 6
Target 99: index -1

[Done] exited with code=0 in 0.142 seconds
```

Justification & Technical Explanation:
Binary search operates strictly on sorted arrays by repeatedly halving the search space. Calculating `mid = low + (high - low) // 2` avoids potential integer overflow bugs in large-scale systems. Best-case time is O(1) if the target resides at the initial midpoint; average and worst-case time is O(log n). Since this is an iterative implementation using two pointers, extra auxiliary space is O(1).

Task 3 – Inventory Management System

Question / Objective:
Search thousands of products by ID/name and sort by price/stock quantity. Recommend and justify algorithms considering dataset size, update frequency, and performance.

AI Prompt:
"Implement product ID lookup, case-insensitive name search, and sorting by price/quantity. Add sample data, docstrings, complexity notes, and an algorithm selection table."

Source Code:

```
def search_by_id(products, product_id):
    """Average O(1) dictionary lookup."""
    return products.get(product_id)


def search_by_name(products, name):
    """Case-insensitive exact-name search; O(n)."""
    target = name.strip().casefold()
    return [p for p in products.values() if p["name"].casefold() == target]


def sort_inventory(products, field="price", descending=False):
    """Sort by price or quantity in O(n log n)."""
    if field not in {"price", "quantity"}:
        raise ValueError("field must be 'price' or 'quantity'")
    return sorted(products.values(), key=lambda p: p[field], reverse=descending)


if __name__ == "__main__":
    inventory = {
        "P101": {"id": "P101", "name": "Notebook", "price": 45.0, "quantity": 120},
        "P102": {"id": "P102", "name": "Pen", "price": 10.0, "quantity": 500},
        "P103": {"id": "P103", "name": "Bag", "price": 650.0, "quantity": 25},
        "P104": {"id": "P104", "name": "Notebook", "price": 55.0, "quantity": 80},
    }
    print("Search by ID:", search_by_id(inventory, "P103"))
    print("Search by name:", search_by_name(inventory, "Notebook"))
    print("By price:", sort_inventory(inventory, "price"))
    print("By quantity:", sort_inventory(inventory, "quantity", True))
```

Terminal Output (VS Code Execution):

```
[Running] python -u "c:\Users\drona\Documents\AI_Assisted_Coding\Lab12\task3.py"
Search by ID: {'id': 'P103', 'name': 'Bag', 'price': 650.0, 'quantity': 25}
Search by name: [{'id': 'P101', 'name': 'Notebook', 'price': 45.0, 'quantity': 120}, {'id': 'P104',
'name': 'Notebook', 'price': 55.0, 'quantity': 80}]
By price: [{'id': 'P102', 'name': 'Pen', 'price': 10.0, 'quantity': 500}, {'id': 'P101', 'name':
'Notebook', 'price': 45.0, 'quantity': 120}, {'id': 'P104', 'name': 'Notebook', 'price': 55.0,
'quantity': 80}, {'id': 'P103', 'name': 'Bag', 'price': 650.0, 'quantity': 25}]
By quantity: [{'id': 'P102', 'name': 'Pen', 'price': 10.0, 'quantity': 500}, {'id': 'P101', 'name':
'Notebook', 'price': 45.0, 'quantity': 120}, {'id': 'P104', 'name': 'Notebook', 'price': 55.0,
'quantity': 80}, {'id': 'P103', 'name': 'Bag', 'price': 650.0, 'quantity': 25}]

[Done] exited with code=0 in 0.175 seconds
```

Justification & Technical Explanation:
Hash table indexing via Python dictionary keys ensures instant O(1) average lookup by unique product ID. Searching across arbitrary product names requires an O(n) linear scan across records unless an inverted hash index is maintained. Sorting uses Python's built-in Timsort (`sorted()`), achieving stable O(n log n) time and O(n) space, ideal for multi-attribute inventory views.

Task 4 – Smart Hospital Patient Management System

Question / Objective:
Search patient records by ID/name and sort by severity/bill amount. Recommend and justify algorithms, then implement them in Python.

AI Prompt:
"Build patient ID/name search and sorting by severity or bill amount; include sample patient records and complexity documentation."

Source Code:

```
def search_patient_by_id(patients, patient_id):
    """Average O(1) lookup by patient ID."""
    return patients.get(patient_id)


def search_patient_by_name(patients, name):
    """Return exact case-insensitive matches; O(n)."""
    target = name.strip().casefold()
    return [p for p in patients.values() if p["name"].casefold() == target]


def sort_patients(patients, field="severity", descending=True):
    """Sort by severity or bill_amount in O(n log n)."""
    if field not in {"severity", "bill_amount"}:
        raise ValueError("field must be 'severity' or 'bill_amount'")
    return sorted(patients.values(), key=lambda p: p[field], reverse=descending)


if __name__ == "__main__":
    patients = {
        "H101": {"id": "H101", "name": "Asha Rao", "severity": 5, "admission_date": "2026-10-12",
"bill_amount": 18000},
```

```
        "H102": {"id": "H102", "name": "Rahul Das", "severity": 2, "admission_date": "2026-10-
10", "bill_amount": 6500},
        "H103": {"id": "H103", "name": "Asha Rao", "severity": 4, "admission_date": "2026-10-13",
"bill_amount": 12000},
    }
    print("Search ID:", search_patient_by_id(patients, "H102"))
    print("Search name:", search_patient_by_name(patients, "Asha Rao"))
    print("By severity:", sort_patients(patients, "severity"))
    print("By bill:", sort_patients(patients, "bill_amount"))
```

Terminal Output (VS Code Execution):

```
[Running] python -u "c:\Users\drona\Documents\AI_Assisted_Coding\Lab12\task4.py"
Search ID: {'id': 'H102', 'name': 'Rahul Das', 'severity': 2, 'admission_date': '2026-10-10',
'bill_amount': 6500}
Search name: [{'id': 'H101', 'name': 'Asha Rao', 'severity': 5, 'admission_date': '2026-10-12',
'bill_amount': 18000}, {'id': 'H103', 'name': 'Asha Rao', 'severity': 4, 'admission_date': '2026-10-
13', 'bill_amount': 12000}]
By severity: [{'id': 'H101', 'name': 'Asha Rao', 'severity': 5, 'admission_date': '2026-10-12',
'bill_amount': 18000}, {'id': 'H103', 'name': 'Asha Rao', 'severity': 4, 'admission_date': '2026-10-
13', 'bill_amount': 12000}, {'id': 'H102', 'name': 'Rahul Das', 'severity': 2, 'admission_date':
'2026-10-10', 'bill_amount': 6500}]
By bill: [{'id': 'H101', 'name': 'Asha Rao', 'severity': 5, 'admission_date': '2026-10-12',
'bill_amount': 18000}, {'id': 'H103', 'name': 'Asha Rao', 'severity': 4, 'admission_date': '2026-10-
13', 'bill_amount': 12000}, {'id': 'H102', 'name': 'Rahul Das', 'severity': 2, 'admission_date':
'2026-10-10', 'bill_amount': 6500}]

[Done] exited with code=0 in 0.171 seconds
```

Justification & Technical Explanation:
A dictionary provides average O(1) patient ID lookup, essential for rapid emergency triage. Name filtering runs in O(n) because each record must be compared. Sorting by medical severity or billing amounts runs in O(n log n); configuring descending=True prioritizes critical patients (severity 5) ahead of routine admissions, enabling responsive clinical workflows.

Task 5 – University Examination Result Processing System

Question / Objective:
Search student results by roll number and sort students by marks for rank lists. Identify, justify, and implement efficient algorithms.

AI Prompt:
"Use a dictionary keyed by roll number for lookup and sorted() for a descending marks rank list. Include sample students and print ranked results."

Source Code:

```
def search_result_by_roll(results, roll_number):
    """Average O(1) lookup by unique roll number."""
    return results.get(roll_number)


def generate_rank_list(results):
    """Sort by marks, highest first; O(n log n)."""
```

```
    return sorted(results.values(), key=lambda s: s["marks"], reverse=True)


if __name__ == "__main__":
    results = {
        "SRU101": {"roll": "SRU101", "name": "Anil", "subject": "Python", "marks": 91},
        "SRU102": {"roll": "SRU102", "name": "Bina", "subject": "Python", "marks": 85},
        "SRU103": {"roll": "SRU103", "name": "Charan", "subject": "Python", "marks": 96},
        "SRU104": {"roll": "SRU104", "name": "Divya", "subject": "Python", "marks": 85},
    }
    print("Search result:", search_result_by_roll(results, "SRU102"))
    print("Rank list:")
    for rank, student in enumerate(generate_rank_list(results), start=1):
        print(rank, student["roll"], student["name"], student["marks"])
```

Terminal Output (VS Code Execution):

```
[Running] python -u "c:\Users\drona\Documents\AI_Assisted_Coding\Lab12\task5.py"
Search result: {'roll': 'SRU102', 'name': 'Bina', 'subject': 'Python', 'marks': 85}
Rank list:
1 SRU103 Charan 96
2 SRU101 Anil 91
3 SRU102 Bina 85
4 SRU104 Divya 85

[Done] exited with code=0 in 0.151 seconds
```

Justification & Technical Explanation:
Because roll numbers are distinct primary keys, hash map indexing provides O(1) average lookup time when students check examination marks. Compiling the merit ranking sorts records by score descending in O(n log n) time using Timsort. Ties (such as Bina and Divya scoring 85) are handled stably in their insertion sequence.

Task 6 – Online Food Delivery Platform

Question / Objective:
Find orders by order ID and sort orders by delivery time or price. Recommend optimized algorithms, justify the choice, and implement search/sort modules.

AI Prompt:
"Implement dictionary order lookup and sorting by delivery time/price, with sample data, field validation, and complexity notes."

Source Code:

```
def search_order(orders, order_id):
    """Average O(1) lookup by order ID."""
    return orders.get(order_id)


def sort_orders(orders, field="delivery_time", descending=False):
    """Sort by delivery_time or price in O(n log n)."""
    if field not in {"delivery_time", "price"}:
        raise ValueError("field must be 'delivery_time' or 'price'")
    return sorted(orders.values(), key=lambda o: o[field], reverse=descending)
```

```
if __name__ == "__main__":
    orders = {
        "O501": {"order_id": "O501", "restaurant": "Spice Hub", "delivery_time": 35, "price":
280, "status": "Preparing"},
        "O502": {"order_id": "O502", "restaurant": "Green Bowl", "delivery_time": 20, "price":
340, "status": "Out for delivery"},
        "O503": {"order_id": "O503", "restaurant": "Pizza Point", "delivery_time": 45, "price":
520, "status": "Confirmed"},
    }
    print("Search order:", search_order(orders, "O502"))
    print("By delivery time:", sort_orders(orders, "delivery_time"))
    print("By price high-to-low:", sort_orders(orders, "price", True))
```

Terminal Output (VS Code Execution):

```
[Running] python -u "c:\Users\drona\Documents\AI_Assisted_Coding\Lab12\task6.py"
Search order: {'order_id': 'O502', 'restaurant': 'Green Bowl', 'delivery_time': 20, 'price': 340,
'status': 'Out for delivery'}
By delivery time: [{'order_id': 'O502', 'restaurant': 'Green Bowl', 'delivery_time': 20, 'price': 340,
'status': 'Out for delivery'}, {'order_id': 'O501', 'restaurant': 'Spice Hub', 'delivery_time': 35,
'price': 280, 'status': 'Preparing'}, {'order_id': 'O503', 'restaurant': 'Pizza Point',
'delivery_time': 45, 'price': 520, 'status': 'Confirmed'}]
By price high-to-low: [{'order_id': 'O503', 'restaurant': 'Pizza Point', 'delivery_time': 45, 'price':
520, 'status': 'Confirmed'}, {'order_id': 'O502', 'restaurant': 'Green Bowl', 'delivery_time': 20,
'price': 340, 'status': 'Out for delivery'}, {'order_id': 'O501', 'restaurant': 'Spice Hub',
'delivery_time': 35, 'price': 280, 'status': 'Preparing'}]

[Done] exited with code=0 in 0.163 seconds
```

Justification & Technical Explanation:

Hash table mapping enables instantaneous O(1) order tracking by unique order ID. Dynamically sorting delivery lists by estimated courier arrival time or ticket value executes in O(n log n) using Timsort, providing high throughput and smooth UI filtering for dispatch controllers.